

Job Shop Scheduling: MIP vs CP

Marta Costa^[202207879], Beatriz Moreira^[202208293], and Beatriz
Sonnemberg^[202206098]

Faculdade de Engenharia da Universidade do Porto

Abstract. The Job Shop Scheduling Problem (JSP) and its variant, the Open Shop Scheduling Problem (OSP), are classical combinatorial optimization problems with high computational complexity and strong practical relevance in industrial environments. This work presents two exact approaches for modeling and solving both JSP and OSP: a Mixed Integer Programming model and a Constraint Programming model. The proposed formulations are evaluated on Taillard benchmark instances to analyze the impact of technological constraints and problem size on solver performance. The experimental study compares solution quality, measured by the makespan, and computational performance in terms of resolution time and scalability. The results reveal clear differences between the two paradigms, showing that constraint-based modeling is particularly effective in handling the disjunctive structure of scheduling problems, especially when contrasting the rigid operation sequences of JSP with the increased flexibility of OSP.

Keywords: Job Shop Scheduling · Mixed Integer Programming · Constraint Programming · Makespan Minimization

1 Introduction

The Job Shop Scheduling Problem (JSP) and the Open Shop Scheduling Problem (OSP) consist of assigning operations to machines under precedence and capacity constraints with the objective of minimizing the makespan. While the JSP imposes a fixed technological order on the operations of each job, the OSP relaxes this requirement, allowing operations to be processed in any sequence and thus increasing scheduling flexibility.

Both problems are NP-hard due to the interaction between machine capacity constraints and operation timing. Despite this complexity, exact approaches are useful for evaluating solver behavior and highlighting performance differences across modeling paradigms.

In this work, JSP and OSP are modeled and solved using two exact optimization paradigms: Mixed Integer Programming (MIP) and Constraint Programming (CP). The MIP formulation relies on continuous start-time variables and binary variables to represent disjunctive machine constraints, whereas the CP formulation exploits interval variables and global constraints to model resource conflicts more compactly.

The proposed models are implemented and experimentally evaluated on Tailard benchmark instances. The analysis focuses on solution quality, computational efficiency, and scalability, highlighting the contrasting behavior of MIP and CP when addressing rigid job sequences in JSP and the increased flexibility of OSP.

2 Problem Description

The scheduling problem considered in this work consists of a set of jobs $J = \{0, \dots, n-1\}$ that must be processed on a set of machines $M = \{0, \dots, m-1\}$. Each job $i \in J$ is composed of a set of operations indexed by $k \in O_i$, where $O_i = \{0, \dots, n_i-1\}$.

Each operation (i, k) has a fixed processing time p_{ik} and must be processed on a specific machine $m_{ik} \in M$. Machines can process at most one operation at a time, and preemption is not allowed.

Two variants of the scheduling problem are considered:

Job Shop Scheduling Problem (JSP): Each job follows a strict technological sequence. Operation (i, k) must be completed before operation $(i, k+1)$ can start.

Open Shop Scheduling Problem (OSP): There is no predefined order for the operations within a job. Operations can be processed in any order, but operations belonging to the same job cannot overlap in time.

The objective in both variants is to minimize the makespan $C_{\max}$, defined as the completion time of the last operation.

2.1 Decision Variables

The MIP formulation uses the following decision variables:

- $s_{ik} \in \mathbb{R}^+$: start time of operation (i, k) .
- $C_{\max} \in \mathbb{R}^+$: makespan of the schedule.
- $y_{ik,jl} \in \{0, 1\}$: binary variable equal to 1 if operation (i, k) is scheduled before operation (j, l) on the same machine.
- $z_{ik,iq} \in \{0, 1\}$: binary variable used in the Open Shop variant, equal to 1 if operation (i, k) precedes (i, q) within the same job.

2.2 Objective Function

The objective is to minimize the makespan:

$$\min C_{\max}$$

2.3 Constraints

Makespan Definition The makespan must be greater than or equal to the completion time of every operation:

$$C_{\max} \geq s_{ik} + p_{ik} \quad \forall i \in J, k \in O_i$$

2.4 Machine Disjunctive Constraints

Each machine can process at most one operation at a time. For every pair of operations (i, k) and (j, l) such that $m_{ik} = m_{jl}$ and $(i, k) \neq (j, l)$, the following Big-M constraints are imposed:

$$s_{ik} + p_{ik} \leq s_{jl} + M(1 - y_{ik,jl})$$

$$s_{jl} + p_{jl} \leq s_{ik} + M(y_{ik,jl})$$

The constant M must be sufficiently large to deactivate one of the constraints depending on the value of the binary variable. In this work, M is set to the planning horizon, defined as the sum of all processing times in the instance. This choice guarantees feasibility while reducing numerical instability compared to arbitrarily large constants.

Job Constraints: Job Shop For the Job Shop variant, a strict technological sequence is enforced:

$$s_{i,k+1} \geq s_{ik} + p_{ik} \quad \forall i \in J, k \in \{0, \dots, |O_i| - 2\}$$

Job Constraints: Open Shop In the Open Shop variant, there is no predefined order between operations of the same job. However, operations within a job cannot overlap. For every pair of operations (i, k) and (i, q) with $k < q$, the following disjunctive constraints are introduced:

$$s_{ik} + p_{ik} \leq s_{iq} + M(1 - z_{ik,iq})$$

$$s_{iq} + p_{iq} \leq s_{ik} + M(z_{ik,iq})$$

3 Methodology and Solution Approaches

To solve the scheduling problems described above, two distinct optimization paradigms are implemented and compared.

3.1 Mixed-Integer Programming (MIP):

Implemented using the PuLP library, this approach follows a classical disjunctive formulation based on Big-M constraints.

A key limitation of the MIP formulation lies in the number of binary variables required to model machine conflicts. For each machine, a binary variable is introduced for every unordered pair of operations processed on that machine. As a result, the number of binary variables grows quadratically with the number of operations per machine.

This rapid growth significantly affects the scalability of the model, increasing both the search space explored by the solver and the difficulty of proving optimality within reasonable time limits.

3.2 Constraint Programming (CP):

Implemented using Google’s OR-Tools CP-SAT solver, this approach models each operation as an interval variable and relies on global constraints such as *NoOverlap*. Unlike MIP, CP avoids explicit pairwise disjunctions and instead exploits constraint propagation and specialized scheduling primitives, resulting in more compact models and improved scalability.

Search Strategies The CP-SAT solver supports different search strategies, which can significantly influence performance. In addition to the default portfolio-based search, the implementation allows for:

- A fixed search strategy, where start time variables are selected in a predefined order and assigned their minimum feasible values.
- A randomized portfolio strategy, which introduces controlled randomness to explore diverse regions of the search space.

These strategies enable an experimental analysis of how search heuristics affect solution quality and computational time.

Compactness and Scalability Compared to the MIP formulation, the CP model is considerably more compact. Each operation is represented by a constant number of variables, and machine conflicts are handled through a single global constraint per machine. As a result, the size of the model grows linearly with the number of operations, rather than quadratically as in the MIP case.

This structural difference has a direct impact on scalability. The CP model can efficiently propagate temporal constraints and prune infeasible schedules early in the search, which often leads to better solution quality and faster convergence for medium and large instances.

4 Experimental Evaluation

This section presents a comprehensive experimental evaluation of the MIP and CP models. The goal of this evaluation is threefold: (i) to validate the correctness of both models, (ii) to compare their performance in terms of solution quality and computational efficiency, and (iii) to analyze their scalability as the problem size increases.

4.1 Experimental Setup

All experiments were conducted using benchmark instances from the Taillard Job Shop Scheduling library, which is widely used in the literature and provides best known solutions (BKS) for makespan minimization. The tested instances range from small to medium sizes, covering configurations from 15×15 up to 30×20 jobs and machines.

To ensure a fair comparison, both models were executed under identical time limits for each instance size. For larger instances, progressively increased time

limits were used in order to balance computational effort with instance difficulty and to analyze solver behavior under controlled computational budgets. Specifically, time limits of 60, 90, 120, 150, and 180 seconds were used for the 15×15 , 20×15 , 20×20 , 30×15 , and 30×20 instances, respectively.

4.2 Model Validation on Small Instances

Before benchmarking large instances, both models were validated on small hard-coded instances with known optimal solutions. For these cases, MIP and CP produced identical makespan values, confirming the correctness of the formulations.

This validation step ensured that any performance differences observed in later experiments are due to solver behavior and model structure rather than modeling errors.

4.3 Solve Time: MIP vs CP

Figure 1 compares the solving time of the MIP and CP approaches for the Taillard 15×15 benchmark instances. The results show a clear and consistent performance gap between the two paradigms.

The MIP solver reaches the imposed time limit of 60 seconds for all tested instances, indicating that even for relatively small problem sizes, the branch-and-bound search struggles to efficiently explore the solution space. This behavior highlights the computational difficulty introduced by the large number of binary decision variables required to model machine disjunctions in the MIP formulation.

In contrast, the CP demonstrates significantly better efficiency. For several instances CP finds feasible solutions well before the time limit. Although CP also reaches the time limit for some instances, it consistently makes better use of the available computational time, reflecting stronger propagation and more effective search heuristics.

Overall, this figure illustrates the superior runtime behavior of Constraint Programming, even at small instance sizes, and anticipates the scalability advantages observed in larger benchmarks.

Additional graphs (Figure 2, 3, 4, 5) have been included to illustrate solver performance for larger instances. As instance size increases, the gap in solving time between MIP and CP tends to decrease, mainly because CP occasionally fails to reach proven optimality while MIP consistently fails to do so.

4.4 Makespan: MIP vs CP

Figure 6 and 10 compare the makespan values obtained by the MIP and CP approaches for smaller and larger instances of the Taillard. Additional intermediate graphs have been included to provide a more detailed view of the solver performance across medium-sized instances (Figure 7, 8 and 9).

For the smaller instances, shown in Figure 6, the CP model consistently achieves makespan values close to the best known solutions, while the MIP model produces substantially higher makespans. Although both solvers operate under the same time limits, the CP is able to exploit problem structure more effectively and converge to high-quality schedules early in the search. In contrast, the MIP solver struggles to significantly improve the incumbent solution once an initial feasible schedule is found.

This performance gap becomes even more pronounced in the larger instances, depicted in Figure 10. As instance difficulty increases, the makespan values returned by the MIP approach grow sharply, indicating large optimality gaps. The CP model, while also affected by the increased complexity, maintains relatively stable makespan values and continues to clearly outperform MIP.

Overall, these figures demonstrate that the superiority of Constraint Programming in terms of solution quality is not limited to small problem instances. The observed difference in makespan persists—and even widens—as the instance size increases, highlighting the better scalability and robustness of CP for job shop scheduling problems under limited computational time.

4.5 Optimality Gap Analysis

To evaluate solution quality, we compare the makespan obtained by each approach (MIP and CP) against the Best Known Solutions (BKS) reported in the Taillard benchmark library.

This metric provides a precise measure of how close each model gets to the global optimum within the allocated time limit. Figure 11 illustrates the average optimality gaps for different instance sizes, highlighting the differences in performance between the MIP and CP formulations.

From the figure, it is evident that the CP model consistently produces schedules closer to the BKS, with substantially lower gaps than the MIP approach.

4.6 Sensitivity Analysis: CP Search Strategies

The performance of the Constraint Programming model strongly depends on the internal search heuristics of the CP-SAT solver. To assess this, we compare three strategies: `DEFAULT`, `FIXED_SEARCH`, and `NO_PRESOLVE`. A single-worker search is used to isolate the effect of each heuristic and ensure a fair comparison.

`DEFAULT` serves as the baseline, leveraging the full SAT-based heuristics of the solver. `FIXED_SEARCH` evaluates the impact of a deterministic variable ordering by disabling dynamic branching, while `NO_PRESOLVE` bypasses initial algebraic simplifications, highlighting the raw complexity of the constraints.

Benchmark instances of size 15×15 , 20×15 , and 20×20 were selected to capture the "transition zone" where the MIP model becomes computationally impractical. This range effectively stress-tests CP, as the search space grows from 225 to 400 operations, allowing us to discern whether performance bottlenecks are due primarily to job sequence length or machine resource contention.

Figure 12, 13 and 14 and shows the solver time for each strategy. It is clear that `FIXED_SEARCH` consistently requires the most time, demonstrating that deterministic branching significantly limits the solver’s ability to efficiently explore the solution space compared to the adaptive heuristics in `DEFAULT`. As the instance size increases, the difference in average time between the various search strategies becomes less pronounced as all approaches eventually converge toward the maximum time limit.

4.7 Open Shop vs Job Shop Analysis

This set of experiments evaluates the impact of relaxing the technological sequence constraints by comparing the Job Shop Problem (JSP) and the Open Shop Problem (OSP). Unlike JSP, OSP allows operations within a job to be executed in any order, provided that no two operations of the same job overlap and no two operations share a machine simultaneously. Mathematically, this relaxation expands the feasible region, so the optimal makespan for OSP is guaranteed to be less than or equal to the corresponding JSP makespan for the same instance.

Flexibility Gain: Allowing the solver to reorder operations increases scheduling flexibility and consistently results in lower makespans. CP exploits this effectively, filling idle machine times that would otherwise be forced by strict sequences.

Modeling Complexity: While OSP improves makespan, it significantly increases the number of binary variables in the MIP model. Each additional disjunctive constraint for sequencing operations within a job doubles the combinatorial complexity, making MIP impractical for medium- and large-sized OSP instances.

CP Scalability: Constraint Programming handles the expanded search space efficiently. Using interval variables and the `AddNoOverlap` constraint, the `DEFAULT` portfolio search strategy navigates the flexible solution space effectively, maintaining high-quality solutions even for larger instances, whereas MIP suffers from the “curse of dimensionality.”

Overall, these experiments demonstrate a clear trade-off: OSP provides better schedules but drastically increases MIP complexity, while CP remains robust and scalable, highlighting the advantages of constraint-based modeling for flexible scheduling scenarios.

5 Conclusion

This study demonstrates that Constraint Programming (CP) consistently outperforms Mixed-Integer Programming (MIP) in solving both Job Shop and Open Shop scheduling problems, particularly as instance size increases.

By exploiting interval variables and global constraints, CP efficiently handles complex temporal and resource conflicts, yielding lower makespans and superior scalability.

Although the Open Shop relaxation provides greater scheduling flexibility and reduces makespan, it substantially increases the complexity of the MIP formulation.

These results reinforce CP as the preferred approach for large or flexible scheduling scenarios.

6 Appendices

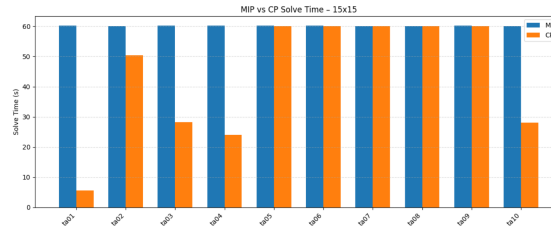


Fig. 1. Comparison of MIP and CP performance

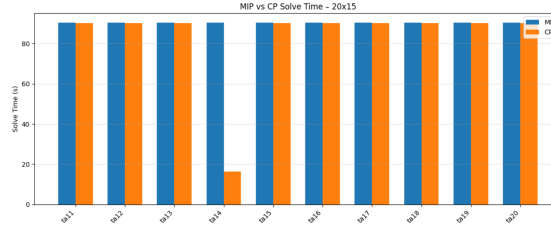


Fig. 2. Solve Time 20x15

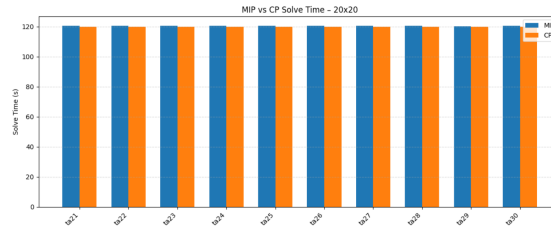
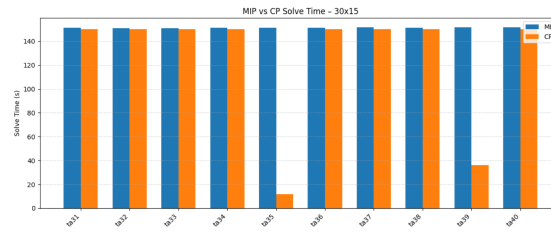
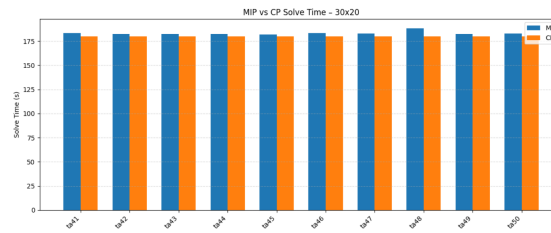
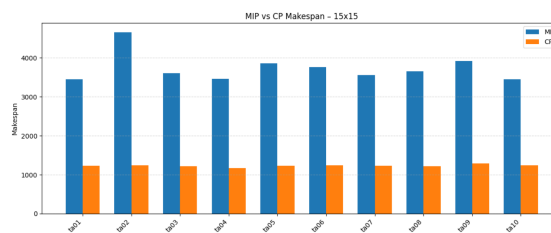


Fig. 3. Solve Time 20x20

**Fig. 4.** Solve Time 30x15**Fig. 5.** Solve Time 30x20**Fig. 6.** Comparison of MIP and CP Makespan - 15x15

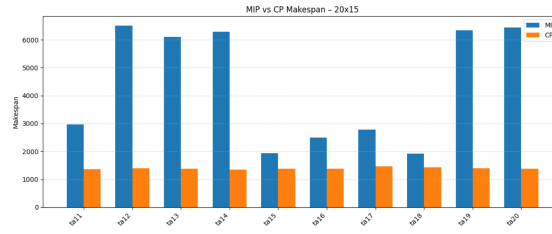


Fig. 7. Comparison of MIP and CP Makespan - 20x15

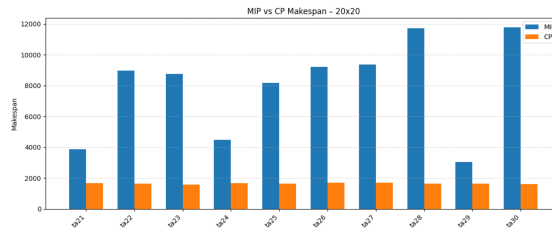


Fig. 8. Comparison of MIP and CP Makespan - 20x20

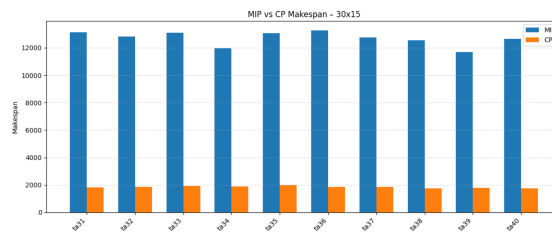


Fig. 9. Comparison of MIP and CP Makespan - 30x15

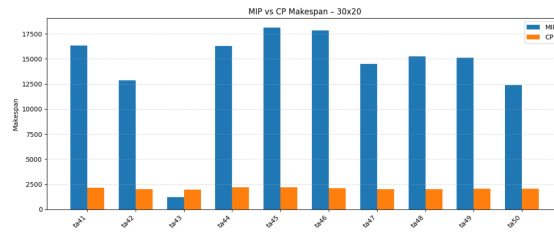


Fig. 10. Comparison of MIP and CP Makespan

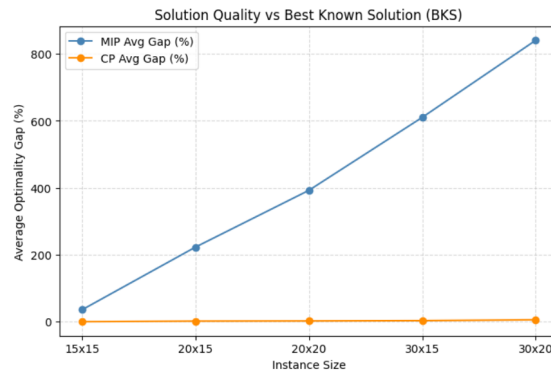


Fig. 11. Solution Quality

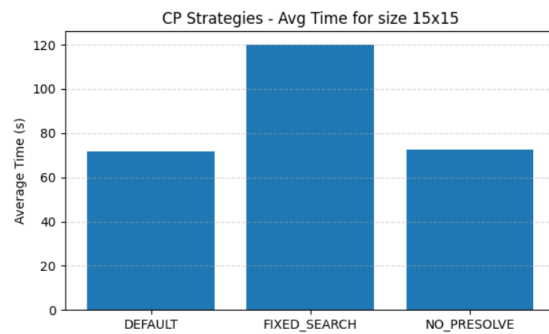


Fig. 12. Avg Time - CP Strategies

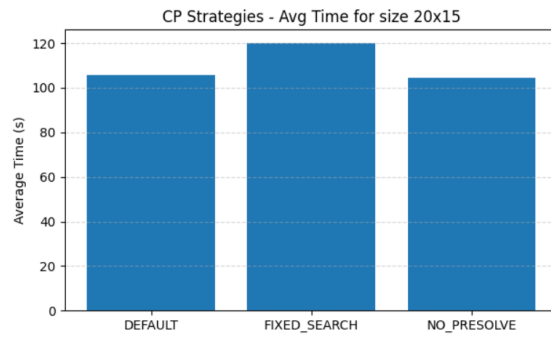


Fig. 13. Avg Time - CP Strategies

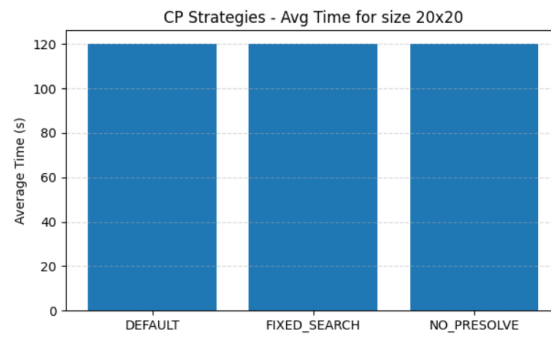


Fig. 14. Avg Time - CP Strategies